Faculty of Information Technology (FIT) - Ho Chi Minh City University of Science (HCMUS)

CS423 / CSC13003 - Software Testing (AI-augmented - 2026)

AI POLICY - TEMPLATES - 2026 v1.0

AI Audit Report

Mandatory appendix for every AI-assisted homework.

1. Student Information

Field	Value
Student name (printed):	NGUYEN THIEN NHA TRAN
Student ID:	23127272
Class / Cohort:	23KTPM2
Assignment ID:	HW06-AI - API Testing on EShop
Assignment date:	18/08/2026
AI tool(s) used:	OpenAI Codex; Context7 documentation connector
AI used in this assignment:	Yes

2. Instructions

This audit records the AI-assisted work behind the current HW06 submission for member 1's API allocation: **Pool A FR-02 login**, **Pool B FR-07 add-to-cart**, and **Pool C FR-15 create-product**. The review basis is `../2026.HW06.API Testing_En.md`: select one API from Pools A, B, and C; generate, audit, extend, and execute at least 40 cases per API; preserve Postman/Newman evidence; report genuine bugs; design a reusable AI-driven test generator; and integrate the suite into CI/CD.

An earlier version of this homework used a different, mis-assigned selection (FR-03/FR-11/FR-14). Because of a human error in the group's API split, the work was redone against member 1's row; the stale artifacts were removed and this audit was rebuilt from the current selection only. The substantive Codex interactions are preserved under `evidence/current-selection-session/`, and the curated prompt/output files are in `../appendix_a/`. Session bootstrap turns, plugin lists, environment context, auto-injected instructions, progress-only messages, tool logs, and hidden reasoning are excluded. Long outputs are referenced by artifact path rather than pasted in full.

3. Audit Table - one row per substantive artifact

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
Tool: Codex **Date:** 2026-08-18 **Prompt:** Read the HW06 specification, build a reusable API-testing generator skill, and begin — but pause for my API selection.	Created the reusable `generate-eshop-api-tests` skill, catalog validator, Postman builder, unit tests, and report/environment/CI scaffolds. Stopped at the selection gate. Evidence: `../skills/generate-eshop-api-tests/`, commit `c860a0e`.	INCOMPLETE	The infrastructure is sound but no test cases could be generated until the three unique Pool A/B/C APIs were chosen.	Provided the three choices (member 1's row) rather than letting the AI invent them.
Tool: Codex **Date:** 2026-08-18 **Prompt:** Take member 1's APIs (FR-02/FR-07/FR-15) and regenerate the catalog, collection, reports, and bug drafts; keep the oracle on the spec.	Regenerated 120 designed cases (40/API): 105 AI cases + 15 student-origin extensions, 119 automated + 1 manual lockout-expiry case. Validator passed; unit tests 3/3. Evidence: `../test-design/test-cases.json`, `../postman/`, `../main-report.md`.	INCOMPLETE	Counts, coverage tags (domain/state/security/schema), and SEC traceability validate, but the AI verdicts are preliminary and the 15 extensions are candidates only. One 30-second boundary case needs a controllable clock.	Student must confirm the VALID/INVALID/INCOMPLETE verdicts, adopt or edit the extensions, and run the manual boundary case.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
Tool: Codex **Date:** 2026-08-18 **Prompt:** Execute the suite deterministically with the X-Student-Id header, and do not report failures as bugs until the harness is proven correct.	The first run was invalid: the build ran from the wrong directory (stale collection) and the SUT on port 3000 could not be reset (Windows file lock). The invalid evidence was discarded; a clean instance on `127.0.0.1:3001` was used to re-run. Evidence: `../automation/run-sut-3001.js`, `../reports/newman-cli.txt`.	INVALID	The initial harness/environment setup produced misleading results that could have been mis-reported as SUT defects. A large failure count means nothing until the harness and environment provenance are verified.	Isolated a clean SUT instance from the pinned commit, corrected the working directory, and re-ran. Final run: 212 requests, 457 assertions, **0** request/pre-request/script failures.
Tool: Codex **Date:** 2026-08-23 **Prompt:** Keep only member 1's version and delete artifacts belonging solely to the earlier FR-03/FR-11/FR-14 selection, without breaking the current version.	Removed the stale artifacts, evidence, PDFs, prior audit, commit log, and builder branches after separating current from stale evidence. Confirmed no residual references and passing validation. Evidence: working-tree state; `../README.md` note.	VALID	The cleanup obeyed the explicit constraint, preserved recoverability through Git history, and left the current selection internally consistent.	None.
Tool: Codex **Date:** 2026-08-29 **Prompt:** Set up Newman-in-GitHub-Actions CI against my fork with a passing run and a one-test-failing run, and package it as a reusable skill; never touch upstream.	Added `.github/workflows/hw06-api.yml` (passing / deliberate-failure modes on the fork) and the reusable `setup-newman-ci-evidence` skill with a deterministic one-failure verifier. Locally: actionlint pass, tests 3/3, verifier 4/4. Evidence: `../.github/workflows/hw06-api.yml`, `../skills/setup-newman-ci-evidence/`, `../ci-cd-report.md`.	INCOMPLETE	The pipeline and verifier are correct locally, but the two required public runs (one green, one red) and their screenshots cannot be produced by AI.	Push and record the passing and deliberate-failure GitHub run links with screenshots.

4. Summary of AI Accuracy

Metric	Count	Percentage
Total AI-generated artifacts audited	5	100%
VALID (correct, accepted as-is)	1	20%
INVALID (wrong; rejected)	1	20%
INCOMPLETE (acceptable after edits/actions)	3	60%

5. Conclusion - When should AI be used (or not)?

Across the five audited artifacts, AI was most useful for repeatable enumeration and automation: deriving domain and boundary partitions for every parameter, expanding state, security (SEC-01-SEC-07), and schema coverage, building a validated catalog and a Postman collection, parsing Newman evidence, and assembling consistent reports and a reusable generator skill. Its strongest failure was in the test harness and environment, not the SUT analysis: the first execution used a stale collection from the wrong directory against an unresettable SUT process, which would have produced misleading failures. This shows that a large failure count is meaningless until prerequisites, working directory, timing, and environment provenance are independently verified. AI also under-weighted cross-request risks - the fast-advancing lockout counter, cart duplicate-merge, client-trusted price/name, and read-after-write persistence - which were strengthened through the student-origin extensions. AI should therefore generate candidate cases, scripts, traceability, and reproducible summaries; it should not be trusted as the final judge of the oracle, harness correctness, human review, or submission completeness. The student must validate the test mechanism, rerun from a clean state, group related symptoms into genuine bugs, preserve raw evidence, and personally create every prohibited or externally attributable artifact.

6. Mandatory Disclosure (paste verbatim)

FR-02, FR-07, and FR-15 test generation, domain and boundary analysis, security and schema traceability, Postman Collection construction, Newman automation, deterministic execution on an isolated SUT instance, result parsing, bug-draft preparation, reusable Agent Skill design, CI/CD pipeline configuration, report drafting, AI Critique drafting, AI Audit organisation, and Appendix A extraction were generated or assisted by OpenAI Codex with documentation lookups through Context7. I reviewed, or am responsible for reviewing, the expected results, preliminary audit verdicts, and final submission contents. The Newman output is a real local run against pinned EShop commit `85af3ba875c88283615e22cb108f13e2fccaf0e9` on `http://127.0.0.1:3001`. Screenshots, public GitHub Issue pages, CI run links, the self-drawn generator diagram, and my signature are not claimed as AI-generated or complete. I confirm I did not use AI to fabricate any artifact listed in the prohibited category.

Signature

Student name (printed):	**NGUYEN THIEN NHA TRAN**
Student ID:	23127272
Class / Cohort:	23KTPM2
Course:	CS423 / CSC13003 - Software Testing
Instructor:	Dr. Lam Quang Vu / Dr. Tran Duy Hoang / MSc. Tran Thi Bich Hanh / MSc. Truong Phuoc Loc / MSc. Ho Tuan Thanh
Date:	30/8/2026
Signature:	Nhã Trân

AI Critique

The AI was effective at expanding a compact specification into systematic domain partitions, boundary values, security traces (SEC-01-SEC-07), JSON-schema assertions, and an executable Postman collection for the login, add-to-cart, and create-product endpoints. Its output was, however, incomplete in two important ways. First, it organised generation mainly around single request-response pairs and under-weighted cross-request behaviour. Cases for the fast-advancing failed-login lockout counter, the cart appending a duplicate row instead of merging quantity, the cart trusting a client-supplied price and name, and read-after-write persistence on product creation were only strengthened after I compared the generated suite against the contract and added student-origin extensions. These behaviours require reasoning about state across several calls, which a prompt centred on parameters and status codes tends to miss. Second, the AI-driven execution harness was initially wrong: the first run used a stale collection from the wrong working directory against a SUT process on port 3000 that could not be reset, producing misleading failures. I caught this by reading the full run instead of trusting the failure count, then isolated a clean SUT instance on port 3001 from the pinned commit and re-ran deterministically, after which the harness reported zero request or script failures. The AI also asserted some status expectations more confidently than the specification justified, so I kept the spec as the oracle and grouped related symptoms into nine genuine bug groups rather than one bug per failed assertion. My main lesson is that collaborating with AI needs two independent reviews - of the test oracle and of the test mechanism. High coverage is not credible when prerequisites, timing, working directory, or evidence provenance are wrong. AI accelerates enumeration and automation, but the student must control state, audit assumptions, reproduce findings, and refuse fabricated evidence.